SelfPlan: Learning Mid-to-Mid Motion Planning through Closed-Map Self-Play

Islem Kobbi[1], Tiago Rocha Gonçalves[12], and Fawzi Nashashibi[1]

***Abstract*—This paper introduces SelfPlan, a closed-map scalable self-play framework for learning mid-to-mid motion planners. The policy outputs a short-horizon trajectory that is tracked by a low-level controller, and it is trained through multi-agent reinforcement learning on closed road networks. The planner uses structured ego-frame observations and a reward combining path quality, efficiency, safety, and goal-reaching terms. Tests on CARLA closed maps, Waymo validation maps, and WOSAC realism metrics show that SelfPlan achieves strong performance, on CARLA maps, it reached 0.97 success, 0.02 collision rate, and 0.01 off-road rate. Without retraining on Waymo maps, it remains competitive with data-driven self-play and obtains the highest efficiency score. It also reaches a 72.7% WOSAC meta-realism score and the best interaction score among the evaluated RL-trained policies. These results indicate that closed-map self-play is a promising scalable direction for learning mid-to-mid motion planners.**

***Index Terms*—Autonomous driving, motion planning, reinforcement learning, self-play, SelfPlan, Gigaflow, mid-to-mid planning.**

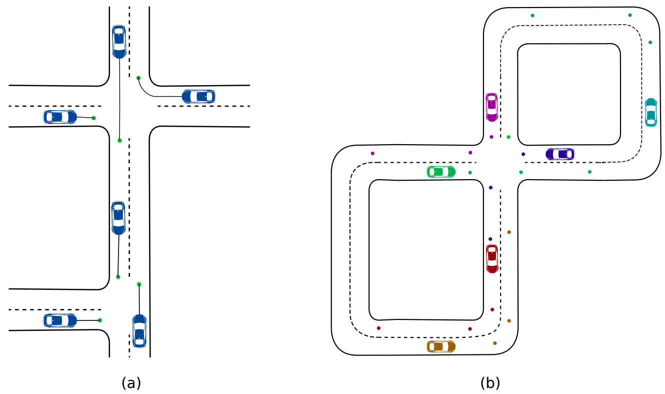

Fig. 1. Comparison between finite log-replay scenes and closed-map self-play. In log replay, the agent is constrained by a finite recorded horizon. In a closed-map setup, agents can keep receiving new goals and generate long-horizon interaction without human demonstrations.

I. Introduction

Motion planning remains a central challenge in autonomous driving. A planner must move safely and efficiently through dense urban scenes while avoiding collisions, staying on the drivable area, following road topology, reaching goals, and interacting naturally with other road users.

End-to-end learning has become an attractive direction for autonomous driving [1], [2], but mid-to-mid planning remains important for systems that require modularity and interpretability [3]–[5]. In this setting, the learned policy outputs an intermediate trajectory or waypoint sequence rather than direct control commands. The plan can then be tracked by a deterministic controller, making the behavior easier to inspect, constrain, and integrate into validation pipelines.

Reinforcement learning (RL) is well suited to this formulation because it optimizes closed-loop behavior through interaction [6]. Unlike pure imitation learning [7], [8], RL lets the agent learn from the consequences of its own decisions and optimize task-level objectives such as progress, goal reaching, and collision avoidance. The main obstacle is scalability: robust driving policies require many simulated interactions, while high-fidelity simulators such as CARLA [9] are costly and data-driven simulators remain limited by dataset coverage [10], [11].

GigaFlow addresses this issue with efficient batched simulation and multi-agent self-play [1]. Agents are trained without human demonstrations on closed road networks, where they repeatedly receive goals and interact with other agents controlled by the same evolving policy. This closed-map design enables long-horizon experience generation beyond the finite duration of recorded logs.

Inspired by GigaFlow's closed-map self-play principle, SelfPlan adapts this idea to mid-to-mid motion planning, where the agent must plan over a longer horizon and anticipate surrounding road users [1], [3], [12]. SelfPlan targets trajectory-level mid-to-mid planning rather than direct control. The evaluation compares log-replay environments, which offer realistic maps but limited coverage, with closed-map self-play environments, which improve scalability but require careful validation against overfitting and distribution shift.

The contributions of this work are:

- **SelfPlan**, a closed-map self-play framework for learning mid-to-mid motion planners;
- a trajectory-level action space based on short-horizon Hermite splines tracked by a low-level controller;
- a reward design emphasizing path quality, efficiency, safety, and goal completion;
- a two-protocol evaluation using closed-loop metrics on CARLA closed maps and Waymo validation maps, together with Waymo Open Sim Agents Challenge realism metrics [13].

[1]I. Kobbi, T. Rocha Gonçalves and F. Nashashibi are with the ASTRA Team, National Institute for Research in Digital Science and Technology (INRIA), 75013 Paris, France. islem.kobbi@inria.fr, tiago.rocha@inria.fr, fawzi.nashashibi@inria.fr

[2]T. Rocha Gonçalves is also with Valeo Brain, 6 rue Daniel Costantini, 94000, Créteil, France. tiago.rocha@valeo.com

II. Related Work

A. RL-Based Motion Planning

Reinforcement learning has been studied for autonomous driving at several levels of abstraction [6], [14]. Mid-to-end formulations map observations directly to control commands [1], [2], [12], while mid-to-mid formulations output an intermediate trajectory or motion primitive executed by a controller [3]. The direct-control formulation is simple, but it merges decision-making and control. In contrast, mid-to-mid planning preserves modularity and makes the planned motion easier to inspect, constrain, or reject before execution.

RL is especially useful when the objective is closed-loop performance [6], [15]. Unlike open-loop imitation [7], [8], which may fail after distribution shift, closed-loop RL exposes the policy to its own errors and optimizes outcomes such as progress, success, collision avoidance, and off-road avoidance.

B. Simulation Speed and Sample Efficiency

A major obstacle for RL in autonomous driving is sample complexity [6]. Physics-based simulators such as CARLA [9] and MetaDrive [16] provide rich scenes and dynamics, but large-scale RL requires more transitions than they can easily generate in real time. This has motivated data-driven replay, GPU-accelerated simulation [10], [11], and learned world models. These approaches trade off realism and scalability for efficiency, and simulation speed.

C. Self-Play for Interactive Driving

Self-play addresses the limitation of fixed background traffic by allowing many agents to share a continuously updated policy [1], [17]. In multi-agent driving, this creates a natural curriculum: early policies learn basic lane following and collision avoidance, while later policies must handle more competent surrounding agents. Compared with scripted traffic or pure log replay, self-play can generate richer and longer-horizon interactions.

GigaFlow is an extreme example of this principle [1]. It trains driving policies from self-play in synthetic closed road networks, without expert demonstrations or logged trajectories. SelfPlan adapts that motivation to a practical question: whether closed-map self-play can provide useful mid-to-mid motion-planning behavior.

III. SelfPlan: Closed-Map Self-Play for Mid-to-Mid Planning

SelfPlan is a reinforcement-learning framework for mid-to-mid motion planning. Instead of outputting low-level control commands, the policy predicts a short-horizon trajectory in the ego frame. This trajectory is tracked by a deterministic controller. Training is performed in closed maps with multiple agents controlled by the same shared policy, allowing the environment to generate long-horizon interaction through repeated goal resampling.

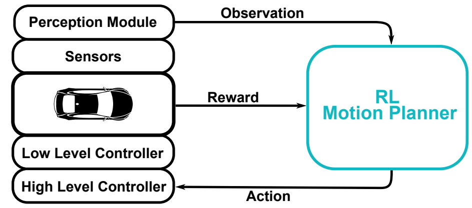

Fig. 2. Overview of the system pipeline.

A. MDP Setup

Mid-to-mid motion planning is formulated as a Markov decision process (MDP)

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \tag{1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition function induced by the simulator and low-level controller, R is the reward function, and γ is the discount factor. At each decision step t, an agent receives an observation $o_t \in \mathcal{O}$ derived from the full simulator state s_t, samples an action $a_t \sim \pi_\theta(\cdot|o_t)$, and obtains a reward r_t. The optimization objective is

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right]. \tag{2}$$

B. Self-Play Rollout and Closed Maps Principle

In each closed map, multiple agents are simulated at the same time and execute the same shared policy. At the beginning of training, the policy produces weak behavior, so interactions are relatively simple. As training progresses, all policy-controlled agents improve together, forcing each ego vehicle to handle increasingly difficult negotiation, avoidance, and overtaking situations. This creates an automatic self-play curriculum without manually designing every traffic scenario.

In the closed-map setup, an episode is not tied to a finite recorded log. Each agent receives a navigation goal and drives toward it from its current position. When the goal is reached, the agent is marked as done for that local task, but the environment is not reset. Instead, a new goal is sampled and the same agent continues driving from its last state. This goal-resampling mechanism allows a limited number of maps to generate many route-following and interaction situations. The agent is reset to its initial position only after terminal failures such as collisions or off-road events. As a result, the policy learns to solve repeated local navigation tasks while maintaining long-horizon safety and progress.

IV. Mid-to-Mid Planner Design

A. Observation Space

The observation follows a structured ego-frame representation divided into ego features, surrounding-agent features, and road-element features. All positions and headings are expressed relative to the controlled agent, which improves invariance to global map coordinates.

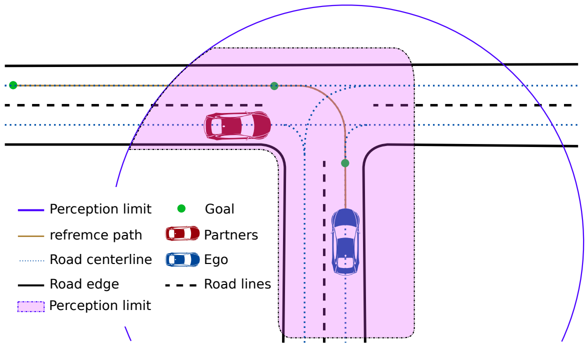

Fig. 3. Structured ego-frame observation used by the mid-to-mid planner.

TABLE I
OBSERVATION LAYOUT.

Component	Max. Count	Dim.	Total
Ego state	1	18	18
ego features	1	8	8
reference path	1	10	10
Road users	31	7	217
Road segments	128	7	896

The ego features encode the relative goal position, target speed, current signed speed, vehicle dimensions, and the reference-path points output by the global route planner [4]. Surrounding road users are represented by local relative position, dimensions, relative heading expressed via sine and cosine, and signed speed. Road elements are represented as local road segments with midpoint, length, width proxy, direction, and semantic type. Table I summarizes the observation layout.

To keep the observation compact and relevant, road elements are selected based on their distance to the reference path, shown by the highlighted region in Fig. 3, rather than only their distance to the ego vehicle. This avoids filling the fixed-size observation vector with less relevant elements, such as road segments that are physically close to the vehicle but lie behind or away from the planned route.

B. Action Space

The mid-to-mid policy outputs a trajectory rather than direct steering and acceleration commands. The action is an eight-dimensional normalized vector,

$$a = [a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7], \tag{3}$$

where each component is scaled and clipped to $[-1, 1]$. The current vehicle state defines the initial control point $P_0 = (0, 0, 0, v_0)$ in the ego frame. The policy then predicts two future control points, P_1 and P_2, with associated positions, headings, and reference speeds, as illustrated in Fig. 4. Let v denote the current speed. The longitudinal scale is defined as

$$\text{scale} = 2 + \frac{v}{4}, \tag{4}$$

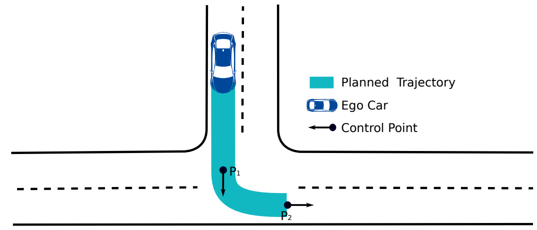

Fig. 4. Eight-dimensional Hermite action. The policy defines two future control points with associated headings and speeds; the resulting spline is tracked by a classical controller.

so that the planning horizon increases with speed. The first control point is decoded as

$$P_{1,x} = (a_0 + 1)\,\text{scale} + \epsilon, \tag{5}$$
$$P_{1,y} = a_1\, d_{\text{max,lat}}, \tag{6}$$
$$h_1 = a_2 \frac{\pi}{2}, \tag{7}$$
$$v_1 = \frac{a_3 + 1}{2} v_{\text{max}}, \tag{8}$$

where $d_{\text{max,lat}} = 10$ m is the maximum lateral displacement, $v_{\text{max}} = 20$ m/s is the maximum reference speed, and ϵ is a small positive longitudinal offset. The second control point is decoded in the local frame of the first control point:

$$P_2 = P_1 + R(h_1) \begin{bmatrix} (a_4 + 1)\,\text{scale} + \epsilon \\ a_5\, d_{\text{max,lat}} \end{bmatrix}, \tag{9}$$

where $R(h_1)$ is the rotation matrix associated with heading h_1. The second heading is

$$h_2 = h_1 + a_6 \frac{\pi}{2}, \tag{10}$$

and the second reference speed is

$$v_2 = \frac{a_7 + 1}{2} v_{\text{max}}. \tag{11}$$

The three control points are interpolated using two cubic Hermite spline segments, $P_0 \to P_1$ and $P_1 \to P_2$. The resulting path is tracked for N_{sub} low-level sub-steps by a controller that uses pure pursuit for steering and PID control for speed.

C. Reward Function

At each decision step, the reward is decomposed into three parts: a path reward, a time reward, and an execution reward accumulated over the low-level controller sub-steps:

$$R_t = R_{\text{path}} + R_{\text{time}} + \sum_{k=1}^{N_{\text{sub}}} R^k_{\text{exec}}. \tag{12}$$

The path reward is evaluated once after decoding the policy action into a Hermite spline. It scores the proposed trajectory before the vehicle executes it. The execution reward is then evaluated during tracking and captures safety, off-road behavior, goal completion, timeout handling, and goal-skipping behavior.

1) Path Quality: The path-quality term evaluates whether the decoded Hermite path is geometrically reasonable before it is executed. It combines route alignment, horizon length, path shape, control-point balance, longitudinal ordering, and temporal continuity:

$$R_{\text{quality}} = c_a S_a + c_l S_l + c_s S_s + c_r S_r + c_o S_o + c_c S_c. \quad (13)$$

The first three terms encourage useful planned motion, while the remaining terms reduce bad control-point configurations and discontinuous replanning.

The *alignment score* S_a measures how closely the two future control points follow the route centerline. For each $i \in \{1, 2\}$, let d_i denote the shortest distance from control point P_i to the centerline, and let α_i denote the heading error with respect to the closest centerline segment:

$$E_a = \frac{c_d(d_1 + d_2) + c_h(\alpha_1 + \alpha_2)}{2}, \quad S_a = 1 - k_a E_a, \quad (14)$$

where c_d and c_h weight the distance and heading errors, and k_a controls the penalty slope.

The *length score* S_l encourages the spline to cover a sufficient horizon for the current speed. If L is the total length of the sampled Hermite path and $L_{\max}$ is the maximum possible length, then

$$S_l = \frac{\min(0.75,\, L/L_{\max})}{0.75}. \quad (15)$$

The score is capped at 75% so that the agent does not favor longer paths at the expense of path quality.

The *symmetry score* S_s compares the lengths of the two Hermite segments, discouraging highly imbalanced control-point layouts that can produce unstable paths:

$$\rho_s = \frac{\min(L_{01}, L_{12})}{\max(L_{01}, L_{12}) + \epsilon}, \quad S_s = 1 - \min\left(1, \frac{\rho_s}{0.25}\right), \quad (16)$$

where L_{01} and L_{12} are the lengths of $P_0 \to P_1$ and $P_1 \to P_2$, respectively. As with the path-length score, the symmetry score is capped once the shorter segment reaches 25% of the longer segment.

The *ratio score* S_r shapes path curvature by rewarding layouts with stronger lateral displacement than longitudinal displacement:

$$r_i = 1 - \min\left(\frac{|P_{i,x}|}{|P_{i,y}| + \epsilon}, 1\right), \; i \in \{1, 2\}, \quad S_r = r_1 r_2. \quad (17)$$

The *ordering score* S_o penalizes trajectories in which the second control point is not farther ahead than the first in the local longitudinal direction:

$$S_o = \begin{cases} \dfrac{P_{1,x} - P_{2,x}}{10}, & \text{if } P_{1,x} > P_{2,x}, \\ 0, & \text{otherwise.} \end{cases} \quad (18)$$

Finally, the *continuity score* S_c discourages sudden changes between consecutive plans. If D_c is the mean point-to-segment distance from the remaining previous path to the new path, then

$$S_c = 1 - k_c D_c, \quad (19)$$

where k_c is the continuity penalty slope.

2) Path Efficiency: The path-efficiency term encourages the planned control-point speeds to match the target speed v_{ref}. This is done with the bounded speed-shaping function

$$\phi(v, v_{\text{ref}}) = \begin{cases} v/(v_{\text{ref}} + \epsilon), & v < v_{\text{ref}}, \\ 1, & v = v_{\text{ref}}, \\ \max(0,\, 1 - (v - v_{\text{ref}})), & v > v_{\text{ref}}. \end{cases} \quad (20)$$

The resulting efficiency reward is

$$R_{\text{eff}} = c_v\, \phi(v_1)\, \phi(v_2), \quad (21)$$

and the complete path reward combines geometric quality and speed efficiency:

$$R_{\text{path}} = w_q R_{\text{quality}} + w_e R_{\text{eff}}. \quad (22)$$

3) Execution Reward: The execution reward evaluates the outcome of tracking the planned path:

$$R_{\text{exec}} = R_{\text{col}} + R_{\text{off}} + R_{\text{goal}} + R_{\text{dnf}} + R_{\text{skip}}. \quad (23)$$

If a collision or off-road event occurs, a hard negative reward is applied and the local rollout terminates for that agent. Otherwise, continuous proximity penalties discourage near-collisions and motion close to the road boundary:

$$R_{\text{col}} = -w_{\text{col}}\left(1 - \min\left(1, \frac{d_{\text{veh}}}{d_{\text{col}}}\right)\right), \quad (24)$$

$$R_{\text{off}} = -w_{\text{off}}\left(1 - \min\left(1, \frac{d_{\text{edge}}}{d_{\text{off}}}\right)\right), \quad (25)$$

where d_{veh} is the nearest road-user distance and d_{edge} is the nearest road-edge distance.

When the agent enters the goal region, it receives a goal reward based on its arrival speed:

$$R_{\text{goal}} = w_{\text{goal}}\, \max(0.5, \phi(v, v_{\text{ref}})). \quad (26)$$

where ϕ is the same function used for the speed score. If the agent does not reach the goal within a fixed number of decision steps, a DNF (*did not finish*) penalty is applied:

$$R_{\text{dnf}} = \begin{cases} -w_{\text{dnf}}, & \text{if } n_{\text{goal}} > N_{\text{goal}}, \\ 0, & \text{otherwise,} \end{cases} \quad (27)$$

where n_{goal} is the number of decision steps elapsed since the current goal was assigned and N_{goal} is the maximum allowed number of steps.

In the SelfPlan closed-map setting, the execution reward also includes timeout and goal-skipping penalties. These terms are important because, in this setup, the simulation does not naturally end. Without them, an agent may learn to drive plausibly while delaying, bypassing, or repeatedly avoiding the navigation objective in order to accumulate additional reward.

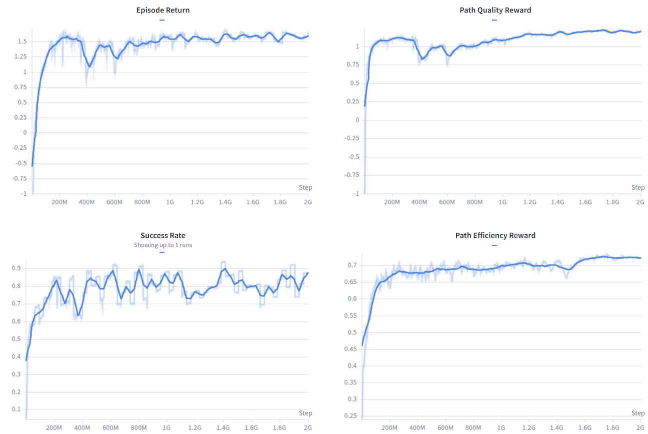

Fig. 5. Training progress over 2 billion environment steps. The four curves summarize return, success rate, path quality, and efficiency during training.

D. Training Setup

Training uses proximal policy optimization (PPO) [18] with a shared policy across all active agents. Each policy decision is executed for multiple simulator sub-steps. In this configuration, $N_{\text{sub}} = 5$ and $\Delta t = 0.1$ s, so one policy action corresponds to 0.5 s of simulated motion. This action-repeat structure reduces the policy frequency while allowing the controller to track the sampled spline over several integration steps, forcing the agent to commit to a planned trajectory at each decision step. The model was trained on the non-layered maps from the CARLA simulator [9] using the GPU-accelerated PufferDrive simulator [11].

V. Experimental Results

Training runs for 2 billion steps. Fig. 5 shows the evolution of key training metrics over the full training run. The policy progressively learns to generate geometrically sound trajectories that reach navigation goals with high centerline alignment, while avoiding collisions and executing smooth overtaking maneuvers around slower agents.

Two evaluation techniques are used to assess the trained agent. The first measures closed-loop task performance using quantitative metrics. The second uses the WOSAC benchmark to assess behavioral realism relative to human drivers. In both evaluations, the model is tested without retraining on the Waymo maps.

A. Metric-Based Closed-Loop Evaluation

The first technique measures closed-loop task performance on two validation domains: the CARLA closed maps on which the agent was trained, and Waymo validation maps. The Waymo domain tests transfer to real-map geometry and evaluates robustness to distributionshift and previously unseen maps. In both domains, a hard-coded planner is used as a reference. On the CARLA maps, the baseline (RB-Baseline) is a rule-based planner. On the Waymo maps, the log-follower planner is designed to reproduce the logged human trajectory as closely as possible. This isolates the degradation caused by the simple controller used for tracking and provides an upper bound for interpreting the learned policies.

Table II reports the quantitative results. On CARLA closed maps, SelfPlan obtains the best success rate of 0.97, reduces the collision rate to 0.02, and maintains a zero DNF rate. It also achieves the highest alignment score in the CARLA setting. These results demonstrate the effectiveness of the method in the closed-map setup, where it substantially outperforms the data-driven self-play policy, which reaches only 0.53 success with a 0.37 DNF rate on the same evaluation.

On the Waymo validation maps, the results show how well SelfPlan transfers from synthetic closed-map training to real-map geometry. The data-driven self-play policy performs slightly better on most metrics, including success rate (0.81 versus 0.78 for SelfPlan), but SelfPlan maintains the highest efficiency score of 0.90.

B. WOSAC Benchmark Evaluation

The second evaluation uses the Waymo Open Sim Agents Challenge (WOSAC) realism metrics [13]. WOSAC evaluates whether simulated agents produce behavior distributions close to real driving in terms of kinematics, interactions, and map-based behavior. This benchmark is complementary to the closed-loop task metrics: a policy can succeed in reaching goals while remaining less realistic, or it can be realistic but insufficiently goal-directed for a specific task.

Table III reports the results. SelfPlan achieves a strong meta-realism score of 72.7%, remaining close to the data-driven self-play model (73.9%) despite being trained in closed-map self-play rather than directly on the WOSAC environment. This result highlights SelfPlan's ability to transfer useful driving behavior beyond its training maps. It also obtains the best interaction score among the evaluated RL-trained policies, showing the effect of self-play.

C. Qualitative Results

Fig. 6 presents a qualitative rollout of the learned planner in a closed-map scenario. The visualization shows that the agent is capable of producing a feasible trajectory, following the road structure, avoiding surrounding traffic, and progressing toward the navigation goal. This example complements the quantitative metrics by illustrating that the learned policy does not only optimize aggregate scores, but also generates interpretable planning behavior during closed-loop execution.

VI. Discussion

The results support the main hypothesis of this work: closed-map self-play can be used to learn effective mid-to-mid motion planners. SelfPlan learns to output short-horizon Hermite trajectories that can be tracked by a conventional controller and that lead to strong closed-loop performance in the CARLA closed-map setting. In particular, SelfPlan reaches a success rate of 0.97, while maintaining low collision and off-road rates. This indicates that the combination of structured ego-frame observations, trajectory-level actions, and reward terms for path quality, efficiency, and execution

TABLE II

CLOSED-LOOP QUANTITATIVE EVALUATION.

Validation Data	Model	Success↑	Collision↓	Off-road↓	DNF↓	Alignment↑	Efficiency↑
CARLA closed maps	RB-Baseline	0.94	0.06	**0.00**	**0.00**	0.83	**0.98**
	Self-play (log replay)	0.53	0.05	0.05	0.37	0.82	0.62
	SelfPlan	**0.97**	**0.02**	0.01	**0.00**	**0.91**	0.97
Waymo validation maps	Log follower	**0.85**	0.08	**0.01**	**0.06**	**0.94**	0.91
	Self-play (log replay)	0.81	**0.06**	**0.01**	0.12	0.64	0.63
	SelfPlan	0.78	0.12	0.04	**0.06**	0.60	**0.90**

TABLE III

WOSAC REALISM EVALUATION.

Method	Meta	Kin.	Inter.	Map
Ground-truth (UB)	0.818	0.607	0.959	0.872
SMART-tiny-CLSFT (SOTA)	0.782	0.520	0.891	0.838
Random	0.446	0.051	0.784	0.470
Log follower	**0.760**	**0.505**	0.879	**0.845**
Self-play	0.739	0.442	0.878	0.795
SelfPlan	0.727	0.425	**0.881**	0.782

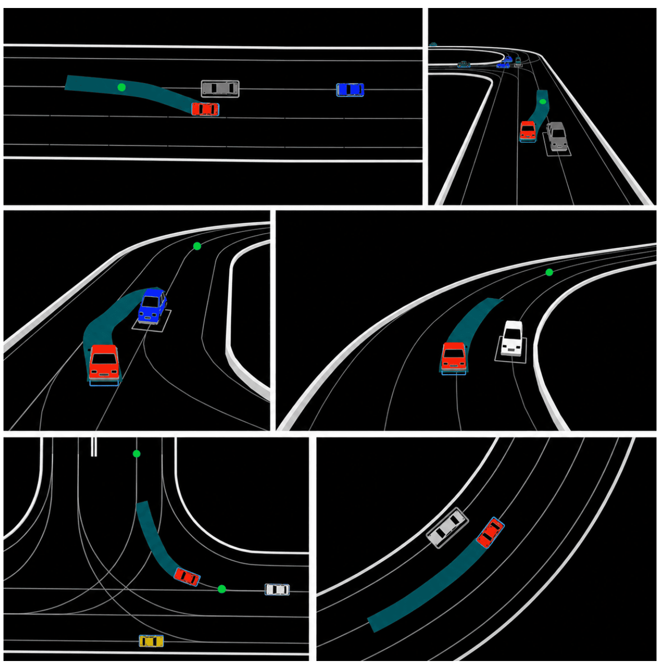

Fig. 6. Qualitative closed-loop example of the learned mid-to-mid planner.

safety is sufficient to produce useful closed-loop planning behavior.

A key advantage of the closed-map formulation is scalability. Because agents repeatedly receive new goals and continue driving after successful local tasks, the environment can generate long-horizon experience without being limited by the duration of a recorded log. Self-play further increases the diversity of interactions because all agents are controlled by the same evolving policy. This makes the training setup useful for rapid experimentation, especially when real closed-loop data are expensive or unavailable. The training curves and CARLA evaluation suggest that this mechanism can transform a limited set of maps into a productive interaction generator for reinforcement learning.

At the same time, the results also show the limits of the current setup. The strong CARLA performance should not be interpreted as full evidence of generalization. The training and evaluation maps are synthetic, and the policy may exploit regularities in map topology, goal sampling, traffic density, or controller behavior. This limitation is visible in the Waymo validation results, where SelfPlan remains competitive but performs slightly below the data-driven self-play policy on most task metrics. The gap indicates that transfer from synthetic closed maps to real-map geometry remains an important challenge for mid-to-mid planning.

The WOSAC evaluation provides a complementary view, showing that SelfPlan can transfer useful driving behavior beyond its synthetic training maps.

These observations motivate a hybrid training strategy. Closed-map self-play can first be used as a scalable pretraining stage to learn goal reaching, lane alignment, collision avoidance, and interaction. The resulting policy can then be fine-tuned or calibrated on real-map or log-replay environments to push performance even further. Future evaluations should also include more CARLA maps, larger and more diverse closed maps, randomized traffic densities, blocked lanes, more complex intersections, and adversarial interaction scenarios, which could further improve the results.

VII. CONCLUSION

This paper introduced SelfPlan, a closed-map self-play framework for learning mid-to-mid motion planning in autonomous driving. The proposed planner uses structured ego-frame observations and outputs a short-horizon Hermite trajectory that is tracked by a low-level controller. The reward function combines path-quality and efficiency terms with execution rewards for collision avoidance, off-road avoidance, goal reaching, timeout handling, and goal-skipping prevention.

The experiments show that closed-map self-play is a promising training paradigm for scalable motion planning. On CARLA closed maps, SelfPlan achieves 0.97 success, 0.02 collision rate, 0.01 off-road rate, zero DNF, and strong alignment and efficiency scores. On Waymo validation maps, the policy remains competitive without retraining, although data-driven self-play performs slightly better on most task metrics. In the WOSAC benchmark, SelfPlan reaches a meta-realism score of 72.7% and achieves the best interaction score among the evaluated RL-trained policies.

Overall, the study shows that scalable self-play can produce useful mid-to-mid planning behavior without relying directly on expert demonstrations. Future work will build on this foundation with richer closed-map training environments and hybrid pretraining–fine-tuning pipelines that combine the scalability of closed-map self-play with the realism of data-driven simulation.

References

[1] M. Cusumano-Towner, D. Hafner, A. Hertzberg, B. Huval, A. Petrenko, E. Vinitsky, E. Wijmans, T. Killian, S. Bowers, O. Sener, P. Krähenbühl, and V. Koltun, "Robust autonomy emerges from self-play," in *Proceedings of the International Conference on Machine Learning*, 2025.

[2] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, "Transfuser: Imitation with transformer-based sensor fusion for autonomous driving," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11. IEEE, 2022, pp. 12 992–13 008.

[3] I. Kobbi, H. Atoui, and F. Nashashibi, "Reinforcement Learning for Mid-to-Mid Motion Planning in Autonomous Driving," in *ITSC 2025 - The IEEE International Conference on Intelligent Transportation Systems*, Gold Coast, Australia, Nov. 2025. [Online]. Available: https://hal.science/hal-05294346

[4] I. Kobbi, F. Nashashibi, and T. Rocha Gonçalves, "Motion planning in urban environments via self-play reinforcement learning," in *Proceedings of the IEEE International Conference on Intelligent Transportation Systems*. Naples, Italy: IEEE Intelligent Transportation Systems Society, Sep. 2026, hAL: hal-05643386.

[5] T. Rocha Gonçalves and L. Guéguen, "RLMP: A sim-to-real reinforcement learning manoeuvre planner validated on public highway," in *The IEEE Intelligent Vehicles Symposium (IV)*, Detroit, United States, Jun. 2026.

[6] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. A. Sallab, S. K. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *CoRR*, vol. abs/2002.00444, 2020. [Online]. Available: https://arxiv.org/abs/2002.00444

[7] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

[8] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.

[9] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the Conference on Robot Learning*, 2017, pp. 1–16.

[10] S. Kazemkhani, A. Pandya, D. Cornelisse, B. Shacklett, and E. Vinitsky, "GPUDrive: Data-driven, multi-agent driving simulation at 1 million FPS," Aug. 2024, p. arXiv:2408.01584.

[11] D. Cornelisse*, S. Cheng*, P. Mandavilli, J. Hunt, K. Joseph, W. Doulazmi, V. Charraut, A. Gupta, J. Suarez, and E. Vinitsky, "PufferDrive: A fast and friendly driving simulator for training and evaluating RL agents," 2025. [Online]. Available: https://github.com/Emerge-Lab/PufferDrive

[12] B. Jaeger, D. Dauner, J. Beißwenger, S. Gerstenecker, K. Chitta, and A. Geiger, "CaRL: Learning Scalable Planning Policies with Simple Rewards," Apr. 2025, p. arXiv:2504.17838.

[13] N. Montali *et al.*, "The waymo open sim agents challenge," in *Proceedings of the NeurIPS Track on Datasets and Benchmarks*, 2023.

[14] B. Paden, M. Cáp, S. Z. Yong, D. S. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *CoRR*, vol. abs/1604.07446, 2016. [Online]. Available: http://arxiv.org/abs/1604.07446

[15] P. Karkus, M. Igl, Y. Chen, K. Chitta, J. A. Packer, B. Douillard, R. Tian, A. Naumann, G. Garcia-Cobo, S. Tan, A. Değirmenci, A. Popov, N. Smolyanskiy, U. Muller, B. Ivanovic, and M. Pavone, "Beyond behavior cloning in autonomous driving: a survey of closed-loop training techniques." [Online]. Available: https://api.semanticscholar.org/CorpusID:283728300

[16] Q. Li, Z. Peng, L. Feng, Z. Zhang, L. Xue, and B. Zhou, "Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3461–3475, 2023.

[17] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017. [Online]. Available: https://doi.org/10.1038/nature24270

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.